Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo

O objetivo desta implementação foi desenvolver um sistema de sincronização para o embarque de passageiros em vagões de metrô, utilizando a linguagem C e a biblioteca Pthreads. O programa deve controlar a disponibilidade de assentos, permitir o embarque simultâneo de passageiros e garantir que o vagão só deixe a estação após a conclusão dos embarques necessários.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Implementar a struct station	OK	Arquivo sync_car.h
Implementação de station_init	OK	Código fonte
Implementação de station_wait_for_car	OK	Teste do runner
Implementação de station_on_board	OK	Teste do runner
Implementação de station_load_car	OK	Teste do runner
Apenas um mutex por estação	OK	Verificação do código
Não utilizar semáforos	OK	Verificação do código
Não utilizar busy-wait	OK	Verificação do sync_car.c
Permitir embarques simultâneos	OK	Teste com múltiplas threads
Compilação por Makefile	OK	Comando make
Gerar executável sync_car	OK	Execução de ./sync_car
Utilização de variáveis de condição	OK	Código-Fonte

3. Como Reproduzir

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

Para repetir a execução 100 vezes e verificar possíveis falhas:

```
for i in $(seq 1 100); do
    ./sync_car > /dev/null || echo "Falha na execução $i"
done
```

4. Arquitetura .

Arquivos e responsabilidades:

1. sync_car.c — Implementa as funções responsáveis pela sincronização dos passageiros e vagões.
2. sync_car.h — Contém a definição da estrutura station e as declarações das funções.
3. sync_car-runner.c — Arquivo fornecido pelo professor para testar o funcionamento da implementação.
4. makefile — Responsável pela compilação e limpeza dos arquivos gerados.
5. README.md — Apresenta as instruções de compilação, execução e teste.
6. evidencias.log — Registra os comandos executados no terminal e seus resultados.

Decisões técnicas:

1. Mutex: utilizado para proteger os contadores compartilhados e evitar alterações simultâneas incorretas.
2. Variáveis de condição: utilizadas para suspender as threads enquanto aguardam assentos ou a conclusão dos embarques, evitando busy-wait.
3. Contadores de passageiros: utilizados para acompanhar passageiros esperando, assentos disponíveis e embarques ainda não concluídos.

5. Estratégias e Diário de Desenvolvimento

5.1 Estratégias da semana

Estratégia	Nome curto	Contexto	
S1	Implementação Constante com testes constantes.	A cada dia da semana ia implementando a funções aos poucos.	Só ia para o próximo passo depois que testava as funcionalidades e dava certo.

5.2 Diário de Tentativas

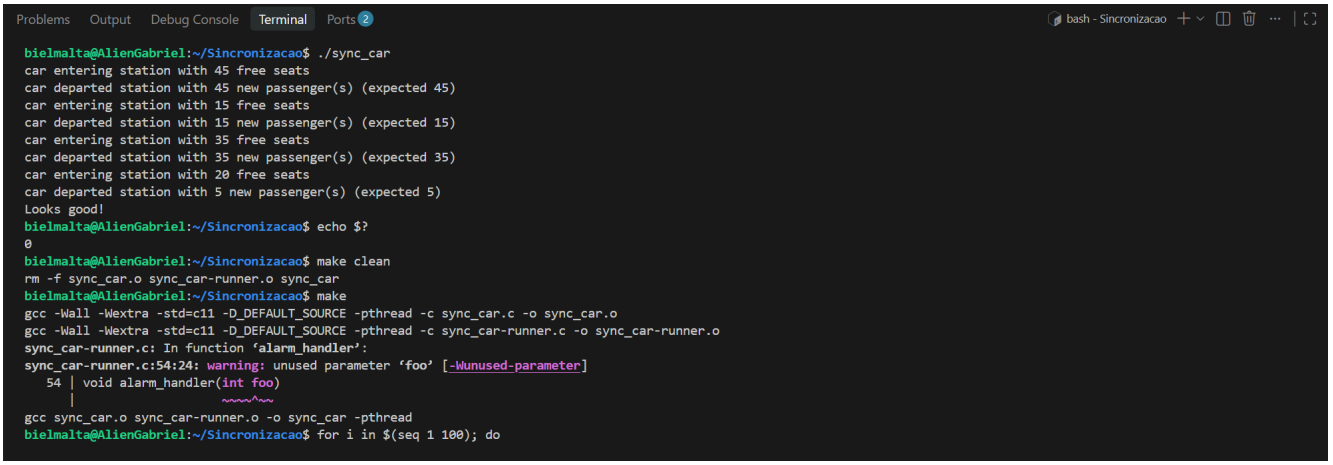
#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência

1	S1	Compilei a primeira versão do programa com make.	Falha	O código ainda não possuía uma função main para gerar o executável.	29/10	Linhas 11–18
2	S1	Compilei apenas o arquivo sync_car.o e testei o make clean.	Sucesso		29/10	Linhas 19–24
3	S1	Compilei a função station_on_board.	Falha	Faltou utilizar & ao passar o mutex e a variável de condição para as funções Pthreads.	29/10	Linhas 46–75
4	S1	Corrigi os argumentos das funções Pthreads e compilei novamente.	Suceso		29/10	Linhas 76–84
5	S1	Compilei o programa com o runner fornecido pelo professor.	Suceso com aviso		03/11	Linhas 101–112
6	S1	Executei o programa para verificar o embarque dos passageiros.	Suceso		03/11	Linhas 113–125
7	S1	Recompilei o programa e executei 100 testes consecutivos.	Suceso		03/11	Linhas 127–149
8	S1	Executei make clean para remover os arquivos compilados.	Suceso		03/11	Linhas 150–152

6. Evidências

Usei o evidencias.log para guardar meus testes e erros que ocorriam ao longo da implementação do projeto, esse arquivo está no arquivo .tar, usei o comando script -a evidencias.log para startar ele, e comecei com os comandos date, whoami e pwd.

6.2 Prints



Validação do embarque dos passageiros

A imagem apresenta a execução do programa utilizando o runner fornecido pelo professor. Foram simulados quatro vagões, que embarcaram 45, 15, 35 e 5 passageiros, respectivamente, totalizando 100 passageiros. A mensagem Looks good! e o código de saída 0 confirmam que o teste foi concluído com sucesso. A imagem também mostra a recompilação do programa e o início do teste de 100 execuções consecutivas.

7. Uso de IA

Utilizei a IA para entender o que a atividade queria que eu fizesse, e também para me explicar todos os conceitos que eu não tinha muito entendimento, como mutex, usando o prompt que o professor nos sugeriu em sala.

“Sou aluno da disciplina de infraestrutura de software e o professor passou essa atividade desgraçada para eu aprender, me ajude com o entendimento dessa atividade. Ele me passou algumas perguntas para me direcionar:

Por que essa atividade existe?

Qual conceito ela quer ensinar?

O que precisa ser entregue?

Quais são os requisitos mínimos?

Como vou dividir o trabalho?

Como vou saber que funcionou?

Para isso, você pode montar um plano de estudos para eu conseguir iniciar os trabalhos para solução da Atividade”.

Também utilizei a IA para revisar meu código, pedindo pra ela me ajudar a entender onde eu errei, e por qual caminho eu deveria seguir. “Revise este código e me dê dicas e insights de onde eu errei e como eu posso ajeitar isso, mas não me dê a resposta, apenas me guie.”

Utilizei a IA também para organizar melhor meu relatório, organizando meu textos e evidencias.

8. Reflexão Final

A implementação ajudou a compreender melhor como mutex e variáveis de condição podem ser utilizados para sincronizar várias threads. Uma das principais dificuldades foi controlar corretamente a quantidade de passageiros esperando e embarcando, sem permitir que o vagão partisse antes do momento correto. Os testes do professor foram importantes para verificar o comportamento do programa em diferentes situações. Com mais tempo, eu realizaria testes adicionais de concorrência, principalmente em situações com muitos passageiros chegando simultaneamente. Também procuraria melhorar a verificação de erros nas chamadas das funções de Pthreads.

9. Checklist Final de Entrega

Item	**Confirmado?**
Compilei do zero seguindo só o que está no relatório	SIM
Rodei os testes e evidencias.log foi gerado	SIM
Tenho prints obrigatórios	SIM
Testei pelo menos um caso limite e um caso inválido	SIM
Preenchi a seção de uso de IA	SIM
Revisei o relatório e removi frases genéricas / vazias	SIM

10. Se eu tivesse mais 2 horas

Se tivesse mais duas horas para trabalhar na implementação, concentraria esse tempo em ampliar os testes de concorrência. Testaria situações com quantidades maiores de passageiros, vagões sem assentos e passageiros chegando durante um embarque. Também verificaria o comportamento do programa em diferentes ordens de execução das threads. Outra melhoria seria adicionar a verificação dos retornos das funções de inicialização e sincronização da biblioteca Pthreads.